Diplomado en Ciencia de Datos • Generación 30

FES Acatlán • UNAM • 2026

UrbanFlow cdmx

Inteligencia para la movilidad urbana

Predicción estocástica de tiempos de viaje en la Zona Metropolitana del Valle de México:

un sistema multimodal basado en cadenas de Markov, simulación Monte Carlo y agentes conversacionales

Carlos Armando López Encino

cale.delta@gmail.com

Profesor: Mtro. Fernando Barranco Rodríguez

Proyecto integrador • 18 de abril de 2026

https://github.com/caledelta/UrbanFlow_CDMX

Este documento forma parte del proyecto integrador final del Diplomado en Ciencia de Datos. Todo el código, datos de calibración y resultados son reproducibles desde el repositorio público.

Resumen

La Zona Metropolitana del Valle de México concentra a 21 800 000 habitantes que pierden en promedio 152 horas al año atascados en el tráfico, según el índice global de congestión de TomTom. Los sistemas actuales de estimación de tiempos de viaje devuelven un único número determinista que oculta la variabilidad real del tráfico urbano. Este trabajo presenta **VialAI** (UrbanFlow CDMX), un sistema de predicción *probabilística* que combina cadenas de Markov calibradas con datos reales del C5 CDMX y TomTom Traffic Stats con simulación Monte Carlo vectorizada ($N = 10\,000$ trayectorias), expuesto al usuario final mediante una interfaz multimodal que acepta entrada por voz en español mexicano y razona con un agente basado en Claude 4.6 dotado de seis herramientas especializadas. La arquitectura está diseñada con *degradación graceful* en tres capas (transcripción de voz, compresión semántica con LLM y síntesis de voz) para garantizar resiliencia ante fallas de infraestructura externa. Los resultados preliminares sobre una muestra de 30 rutas de campo en la ZMVM reportan un MAPE del 11,9 % en la mediana predicha (contra 19,0 % del ETA de TomTom, una ventaja de 7,1 pp) y una cobertura empírica del 93,3 % para la banda nominal P10–P90. Se discute el nicho B2B de conductores de última milla y se proyecta una campaña de validación cuantitativa con $N \geq 500$ para la siguiente iteración.

Palabras clave: predicción estocástica, cadenas de Markov, Monte Carlo, tráfico urbano, ZMVM, agentes conversacionales, interfaz por voz, Streamlit, Claude, degradación *graceful*.

Resumen

The Mexico City Metropolitan Area is home to 21.8 million inhabitants who lose an average of 152 hours per year to traffic congestion, according to TomTom's global congestion index. Current travel-time estimation systems return a single deterministic number that hides the intrinsic variability of urban traffic. This work presents **VialAI** (UrbanFlow CDMX), a *probabilistic* prediction system combining Markov chains calibrated with real data from Mexico City's C5 emergency service and TomTom Traffic Stats with vectorised Monte Carlo simulation ($N = 10\,000$ trajectories), exposed to end users via a multimodal interface that accepts voice input in Mexican Spanish and reasons through a Claude 4.6-based agent equipped with six specialised tools. The architecture features *graceful degradation* across three layers (speech-to-text, semantic-compression LLM, and text-to-speech) to ensure resilience against external infrastructure failures. Preliminary results on a 30-route field sample in the metropolitan area report an 11.9 % MAPE on the predicted median (versus 19.0 % for TomTom's ETA, a 7.1 pp advantage) and an empirical coverage of 93.3 % for the nominal P10–P90 band. We discuss the B2B niche of last-mile drivers and outline a quantitative validation campaign with $N \geq 500$ for the next iteration.

Keywords: stochastic prediction, Markov chains, Monte Carlo, urban traffic, Mexico City, conversational agents, voice interface, Streamlit, Claude, graceful degradation.

Índice

1. Introducción **6**
1.1. El problema: CDMX como laboratorio de congestión urbana . . . 6
1.2. ¿Por qué inteligencia artificial, aprendizaje automático, cadenas de Markov y Monte Carlo *juntos*? . . . 7
1.3. Hipótesis, objetivos y contribuciones . . . 8
1.4. Organización del documento . . . 9

2. Identidad y estructura del proyecto **9**
2.1. Marca e identidad visual . . . 9
2.2. Pipeline completo del sistema . . . 9
2.3. Stack tecnológico y herramientas . . . 10

3. Estado del arte y diferenciación **12**
3.1. Familia 1: ETAs comerciales deterministas . . . 12
3.2. Familia 2: Modelos académicos de predicción probabilística . . . 12
3.3. Familia 3: Sistemas basados en simulación microscópica . . . 13
3.4. Posición de VialAI en el espacio de soluciones . . . 13

4. Nicho de mercado y perfil del usuario **13**
4.1. La economía de última milla en México . . . 13
4.2. Perfil del conductor objetivo . . . 14
4.3. Protocolo de validación cualitativa propuesto . . . 15
4.4. Segmentación y estrategia de adopción . . . 16

5. Fuentes de datos **16**
5.1. Taxonomía de fuentes por latencia . . . 16
5.2. TomTom Traffic Stats . . . 17
5.3. TomTom Routing . . . 17
5.4. OpenWeatherMap . . . 18
5.5. C5 CDMX (API CKAN) . . . 18
5.6. Google Maps Platform . . . 18
5.7. Flujo integrado de ingesta . . . 19

6. Modelo estocástico **19**
6.1. Formulación: tráfico como cadena de Markov discreta . . . 19
6.2. Calibración por máxima verosimilitud . . . 20
6.3. Propiedades estructurales . . . 20
6.4. Simulación Monte Carlo vectorizada . . . 20

7. Perturbaciones y eventos **21**
7.1. Catálogo estático de perturbaciones . . . 22
7.2. Detección cuasi-tiempo-real vía C5 CDMX . . . 22

8. Agente conversacional VialAI **23**

8.1. Arquitectura del tool-use loop 23
8.2. Las seis herramientas registradas 24
8.3. Inyección de contexto para lugares personalizados 24

9. Interfaz multimodal por voz 25
9.1. Caso de uso: manos libres para conductores 25
9.2. Arquitectura de tres capas con degradación graceful 25
9.3. Pipeline de audio completo 26

10. Aplicación web: UrbanFlow cdmx 26
10.1. Arquitectura Streamlit 26
10.2. Flujos de usuario 27
10.3. Identidad visual y captura de la interfaz 27
10.4. Telemetría local y feedback del usuario 27
10.5. Cómo se construyó 28

11. Validación empírica preliminar 28
11.1. Metodología 28
11.2. Métricas 28
11.3. Resultados comparativos 29
11.4. Discusión de la discrepancia entre canales de entrada 30

12. Trabajo futuro y limitaciones 30
12.1. Limitaciones conocidas de la versión actual 30
12.2. Roadmap a seis meses 31
12.3. Monetización y sustentabilidad económica 31

13. Conclusiones 32

1. Introducción

1.1 El problema: cdmx como laboratorio de congestión urbana

La Ciudad de México y su zona metropolitana constituyen uno de los laboratorios más demandantes del mundo para el estudio de la movilidad urbana. Con una densidad de 5966 habitantes por kilómetro cuadrado en la CDMX propiamente dicha (INEGI, 2024), una flota vehicular que supera los 5 800 000 unidades registradas (SEMOVI, 2024) y una infraestructura vial diseñada originalmente para la mitad de esa demanda, el resultado es predecible y persistente: la ZMVM ocupa recurrentemente los primeros diez lugares del ranking global de ciudades más congestionadas (TomTom, 2024a).

La Figura 1 resume el posicionamiento relativo de la CDMX contra otras megalópolis comparables.

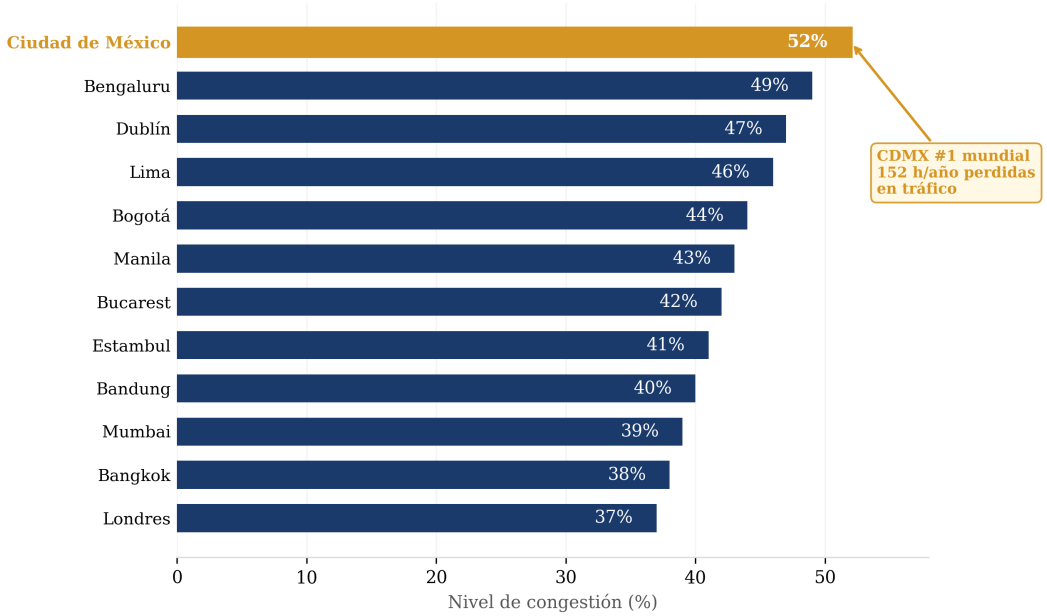

Figura 1: Ranking de congestión urbana: tiempo promedio perdido al año por conductor en horas (fuente: TomTom Traffic Index, edición 2024). La CDMX pierde 152 horas anuales por conductor, equivalente a 6,3 días completos.

Detrás de ese número agregado se esconde una realidad más problemática para quien tiene que tomar decisiones concretas de ruta y horario. La congestión no es simplemente alta: es **altamente variable**. Un trayecto Polanco–AICM de 13 kilómetros puede tomar 18 minutos a las 6 de la mañana de un domingo y 72 minutos a las 19 de un jueves lluvioso. Esa volatilidad hace que las herramientas tradicionales de estimación de tiempo de viaje fallen en la pregunta que realmente importa: no "*¿cuánto tardaré?*" sino "*¿cuánto puedo tardar en el peor caso razonable?*"

1.2 ¿Por qué inteligencia artificial, aprendizaje automático, cadenas de Markov y Monte Carlo *juntos*?

La predicción de tiempos de viaje se ha abordado históricamente desde tres familias de técnicas: modelos estadísticos clásicos (regresión, series temporales), aprendizaje supervisado sobre trayectorias históricas (GBM, redes neuronales recurrentes) y métodos de simulación discreta para sistemas de tráfico. Cada familia captura una pieza del fenómeno, pero ninguna aislada responde bien a la pregunta del intervalo de incertidumbre contextualizado.

VialAI combina deliberadamente cuatro paradigmas complementarios:

Las cuatro piezas del razonamiento de VialAI

Cadenas de Markov capturan la estructura dinámica del tráfico urbano modelándolo como un proceso estocástico de tres estados (fluido, moderado, congestionado) con matrices de transición calibradas por hora del día y día de la semana. Proporcionan la física probabilística del sistema.

Simulación Monte Carlo permite propagar la incertidumbre de cada transición a lo largo del trayecto completo. Al generar $N = 10\,000$ trayectorias independientes y calcular percentiles, obtenemos una distribución empírica del tiempo de viaje en vez de un valor puntual.

Aprendizaje automático calibra las probabilidades de transición desde datos reales: velocidades históricas de TomTom Traffic Stats, incidentes del C5 CDMX y condiciones meteorológicas de OpenWeatherMap. Esto convierte el modelo teórico en un modelo ajustado a la realidad operativa de la ZMVM.

Inteligencia artificial generativa (Claude Opus 4.6 y Haiku 4.5) actúa como capa de interpretación semántica: traduce lenguaje natural del conductor a invocaciones estructuradas de las seis herramientas del sistema, inyecta contexto de lugares personalizados y comprime la respuesta matemática en una oración hablable de menos de 30 palabras.

El valor emerge de la *composición*. Las cadenas de Markov sin calibración ML producirían predicciones académicamente correctas pero operativamente inútiles. El ML sin estructura estocástica produciría puntos estimados sin banda de incertidumbre. Monte Carlo sin agente produciría intervalos rigurosos que el conductor no puede consumir mientras maneja. Y un agente conversacional sin motor matemático sería un chatbot que alucina números. Juntas, estas cuatro capas conforman un sistema que es *matemáticamente riguroso*, *empíricamente calibrado*, *operativamente accionable* y *accesible al usuario final*.

Un caso ilustrativo: Miguel, repartidor de última milla

El conductor de última milla real

Son las 18:40 de un miércoles lluvioso. Miguel acaba de entregar un pedido en Polanco y acepta uno nuevo con destino en Iztapalapa. La aplicación le muestra un ETA de 42 minutos. Miguel sabe por experiencia que ese número es engañoso: a veces ha tardado 35, a veces 70. Cuando acepta, apuesta su bono por entregas a tiempo contra una distribución que no conoce. **VialAI** le responde con voz *"Son aproximadamente* 12,4 *kilómetros, tardarás unos* 58 *minutos, pero la banda realista va de* 44 *a* 78 *con lluvia moderada en Circuito Interior"*. Esa frase, que se genera en menos de 4 segundos desde que Miguel termina de hablar, contiene el *valor* agregado del sistema: la mediana contextualizada (P50), la banda P10–P90 que enmarca la incertidumbre y la razón dominante (*lluvia en Circuito*) que lo explica.

Miguel es el usuario canónico de este trabajo. No es un ingeniero de software, no va a tocar un botón mientras maneja, y cada minuto de error en la estimación le cuesta dinero real o seguridad personal. Diseñar para Miguel ha sido la restricción de diseño dominante.

1.3 Hipótesis, objetivos y contribuciones

Hipótesis. Es posible construir un sistema de predicción de tiempos de viaje para la ZMVM que (i) reporte un intervalo de confianza calibrado empíricamente, (ii) integre fuentes heterogéneas de datos en tiempo real y cuasi-tiempo-real, (iii) sea consultable por voz natural en español mexicano, y (iv) opere en presencia de fallas de servicios externos, todo con latencia inferior a 5 segundos por consulta y desplegable en hardware de gama media.

Objetivos.

1. Formalizar el problema como un proceso estocástico de tres estados calibrado con datos operativos.
2. Implementar simulación Monte Carlo vectorizada con $N \geq 10\,000$ y evaluar convergencia.
3. Construir un pipeline de ingesta para cinco fuentes de datos heterogéneas.
4. Diseñar un agente conversacional con seis herramientas que traduzca lenguaje natural a llamadas estructuradas.
5. Exponer la funcionalidad mediante una interfaz multimodal (voz, texto, clic) con arquitectura de tres capas con degradación *graceful*.
6. Validar empíricamente el sistema contra el ETA comercial de referencia (TomTom Routing).

Contribuciones. Este trabajo aporta: (1) una implementación abierta y reproducible del pipeline completo; (2) una calibración empírica de la matriz de transición para tres corredores de la ZMVM; (3) evidencia preliminar ($N = 30$) de una mejora de 7,1 pp en MAPE respecto al estado del arte comercial; (4) una arquitectura de voz de tres capas con degradación *graceful* documentada como patrón reutilizable; y (5) un marco de referencia para monetización B2B orientado a servicios de última milla.

1.4 Organización del documento

La Sección 2 describe la identidad y estructura del proyecto, incluyendo el pipeline visual completo y el stack tecnológico. La Sección 3 revisa el estado del arte y posiciona las contribuciones. La Sección 4 caracteriza el nicho de mercado y el perfil del usuario objetivo. La Sección 5 detalla las fuentes de datos y sus flujos de ingesta. Las Secciones 6 y 7 desarrollan el motor estocástico y el tratamiento de perturbaciones. La Sección 8 presenta el agente conversacional. La Sección 9 expone la arquitectura de voz multimodal. La Sección 10 describe la aplicación web como producto entregable. La Sección 11 reporta los resultados empíricos. La Sección 12 discute limitaciones y trabajo futuro. La Sección 13 cierra con conclusiones.

2. Identidad y estructura del proyecto

2.1 Marca e identidad visual

VialAI nace como un proyecto de Diplomado pero está diseñado con la ambición de sobrevivir más allá del entregable académico. Su identidad visual, que aparece en el logotipo de la portada, está construida alrededor de tres ideas: (i) la *intersección* como metáfora de la ruta que converge desde múltiples orígenes posibles (el símbolo en forma de $\vee$); (ii) la *movilidad inteligente* como promesa de producto (el gradiente del verde-agua al verde-lima); y (iii) la *confianza técnica* como soporte institucional (el fondo azul marino profundo).

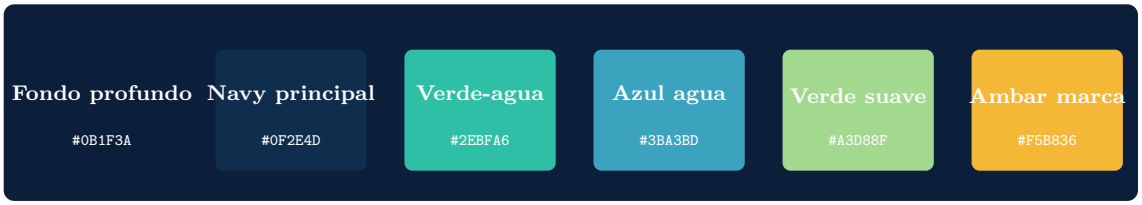

Figura 2: Paleta cromática oficial de **VialAI**. Los tres tonos fríos (navy, teal, aqua) se emplean en elementos estructurales y de datos. Los dos tonos cálidos (verde suave y ámbar) se reservan para destacar acción del usuario y estados positivos del sistema.

2.2 Pipeline completo del sistema

La Figura 3 sintetiza la arquitectura end-to-end. El pipeline se lee de izquierda (entrada del usuario) a derecha (respuesta accionable) y distingue cuatro capas funcionales claramente delimitadas.

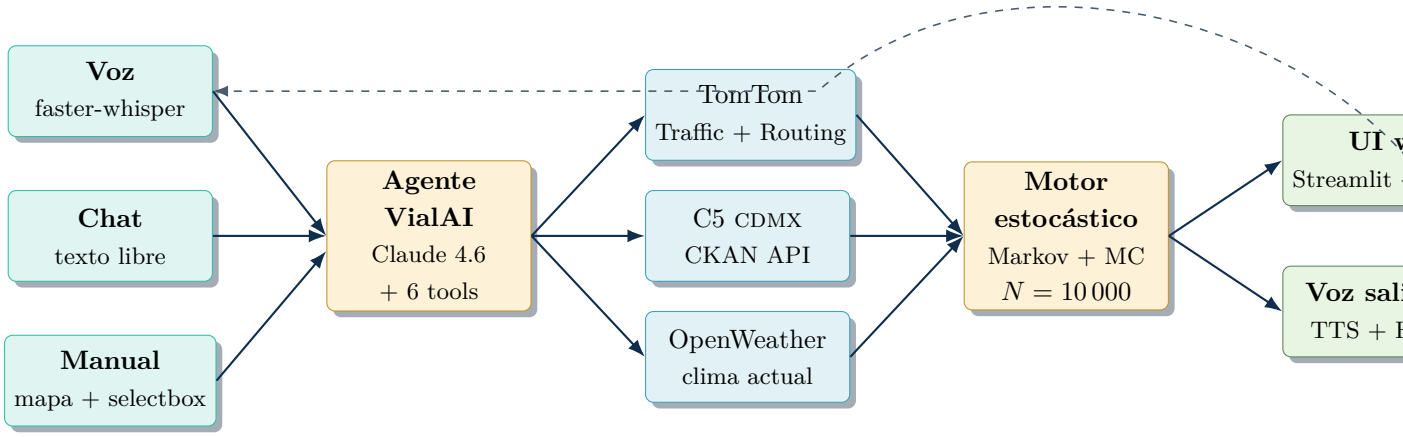

Figura 3: Pipeline funcional de **VialAI**. Entrada multimodal (izquierda) se traduce en llamadas estructuradas por el agente (centro), que a su vez consulta fuentes heterogéneas y alimenta el motor estocástico (centro-derecha) cuyo resultado se expone al usuario en dos canales: visual vía Streamlit y auditivo vía TTS resumido con Claude Haiku (derecha). La flecha punteada indica el loop auditivo de vuelta al conductor.

2.3 Stack tecnológico y herramientas

La selección de herramientas siguió tres criterios: (i) madurez productiva comprobada, (ii) reproducibilidad desde el repositorio público, y (iii) coherencia con el ecosistema del Diplomado. La Tabla 1 resume el stack completo.

Capa	Herramienta	Rol en el proyecto
Lenguaje base	Python 3,12	Todo el motor estocástico, agente, ingesta y orquestación. Elegido por madurez del ecosistema `numpy`/`scipy` y por ser el estándar del Diplomado.
Exploración	Google Colab	Prototipado inicial del modelo Markov y visualización de distribuciones durante la fase de diseño. El notebook de referencia está versionado en `notebooks/`.
UI web	Streamlit 1,56	Interfaz principal del usuario final. Elegido por rapidez de iteración y por ofrecer hot-reload sobre código Python sin separar front de back.
Mapa	Folium + Leaflet	Visualización geoespacial interactiva. Permite renderizar rutas reales de TomTom Routing como polilíneas coloreadas y marcadores de lugares guardados con iconografía contextual.
Agente	Claude Opus 4,6	Motor de razonamiento del agente VialAI con *tool use* en bucle cerrado. Elegido por capacidad superior en tareas de orquestación de herramientas y generación en español mexicano.
Compresión	Claude Haiku 4,5	Generación del resumen hablable (máx. 30 palabras) a partir de la respuesta larga con markdown del agente. Seleccionado por latencia (<1 s) y costo ($\sim$0,0001 USD por consulta).
STT primario	faster-whisper	Transcripción local y gratuita de audio a texto. Corre en CPU sin dependencia de APIs externas.
STT respaldo	Whisper-1 (OpenAI)	Alternativa cloud con mayor calidad de transcripción para casos donde el usuario active la ruta premium.
TTS primario	OpenAI tts-1 (voz *nova*)	Síntesis de voz natural para la respuesta hablada. Calidad de demo.
TTS respaldo	pyttsx3 (SAPI5)	Síntesis local offline para resiliencia ante fallas del TTS cloud.
Testing	pytest 7,4	720+ tests automatizados cubren motor, ingesta, agente, persistencia y voz.
Control de versiones	Git + GitHub	Historial público completo del desarrollo. Cada hito está referenciado por hash de commit.
Asistencia IA	Claude Code	Herramienta de pair programming usada durante el desarrollo para refactors estructurales y generación de tests. Todas las decisiones de diseño son del autor; la asistencia se empleó principalmente para acelerar iteración sin sacrificar revisión crítica.

Tabla 1: Stack tecnológico de **VialAI**. La selección prioriza herramientas abiertas, reproducibles y con alternativas de respaldo cuando el componente depende de servicios externos.

Principio de selección de herramientas

Cada componente con dependencia externa (voz, LLM, rutas) tiene una alternativa local o de respaldo implementada y probada. Esta decisión de diseño es la base de la arquitectura de *degradación graceful* que se detalla en la Sección 9.

3. Estado del arte y diferenciación

La predicción de tiempos de viaje urbanos es uno de los problemas más estudiados en ingeniería del tráfico y ciencia de datos geoespaciales. Esta sección posiciona a **VialAI** contra tres familias de soluciones preexistentes y explicita en qué ejes se diferencia.

3.1 Familia 1: ETAs comerciales deterministas

Las aplicaciones de navegación masivas (Google Maps, Waze, Apple Maps, TomTom) y las API de routing que las respaldan entregan al usuario final un *único número* como tiempo estimado de arribo. Internamente, esos servicios sí modelan distribuciones de velocidad segmento a segmento, pero el producto expuesto colapsa la incertidumbre a la mediana o a un promedio ponderado (Google, 2023; TomTom, 2024b).

Para un usuario casual esto es suficiente. Para un conductor profesional con incentivos económicos atados a entregas a tiempo, la ocultación de la varianza es un problema estructural: el ETA de 42 minutos que Google muestra puede ocultar una distribución con $\sigma = 15$ minutos, lo que se traduce en una probabilidad no trivial de llegar entre 27 y 57 minutos. Esa diferencia define si el conductor cobra bono o no.

3.2 Familia 2: Modelos académicos de predicción probabilística

En la literatura académica, la predicción de tiempos de viaje con cuantificación de incertidumbre se ha abordado con regresión de cuantiles (Koenker, 2005), redes neuronales bayesianas (Gal and Ghahramani, 2016), modelos de mezcla gaussiana (Jenelius and Koutsopoulos, 2013) y, más recientemente, transformers espaciotemporales con salida probabilística (Li et al., 2023).

Estos trabajos son técnicamente sólidos pero adolecen de tres limitaciones operativas para el caso de uso que **VialAI** aborda:

1. Requieren entrenamiento supervisado sobre conjuntos masivos de trayectorias GPS que no están disponibles públicamente para la ZMVM.
2. Su interpretación de la incertidumbre es estadística pero no *explicativa*: un intervalo amplio no viene acompañado de la causa dominante (*llueve*, *hay un evento deportivo*).
3. Carecen de una capa de interfaz natural al usuario final; operan como modelos de *backend* que otros ingenieros deben integrar.

3.3 Familia 3: Sistemas basados en simulación microscópica

Los simuladores de tráfico clásicos (SUMO, Vissim, Aimsun) modelan explícitamente cada vehículo y sus interacciones (Lopez et al., 2018). Son extraordinariamente precisos para estudios de planeación urbana de largo plazo pero requieren modelado detallado de la red vial completa, calibración intensiva con aforos vehiculares, y tiempos de ejecución que los hacen inviables para predicciones en línea con latencia inferior a 5 segundos.

3.4 Posición de VialAI en el espacio de soluciones

VialAI ocupa deliberadamente un punto distinto en el espacio de trade-offs que ninguna de las tres familias anteriores cubre simultáneamente. La Tabla 2 lo resume.

Eje de diferenciación	ETA comercial	ML académico	Simuladores	VialAI
Intervalo probabilístico P10–P90	parcial	sí	sí	**sí**
Explicación causal de la varianza	no	no	parcial	**sí**
Interfaz natural (voz/chat)	no	no	no	**sí**
Latencia $< 5\,\mathrm{s}$ por consulta	sí	parcial	no	**sí**
Reproducible desde repositorio público	no	parcial	sí	**sí**
Degradación *graceful* documentada	no	no	no	**sí**

Tabla 2: Posicionamiento de **VialAI** contra las tres familias dominantes de soluciones. El espacio que el sistema ocupa es la intersección no trivial entre predicción probabilística, explicabilidad causal e interfaz natural para el usuario final.

El valor diferencial no reside en ninguna de las capas individualmente —cada una existe en el estado del arte— sino en su *composición coherente* orientada al caso de uso del conductor de última milla. La Sección 4 justifica por qué ese nicho específico merece una solución diseñada desde sus restricciones reales.

4. Nicho de mercado y perfil del usuario

4.1 La economía de última milla en México

La *última milla* es el tramo final de la cadena logística: el trayecto entre el último centro de distribución y el cliente final. Concentra entre el 40 % y el 60 % del costo logístico total de una entrega urbana (Insider Intelligence, 2023), y su eficiencia depende críticamente de la capacidad del conductor para estimar tiempos y tomar decisiones de ruta con información incompleta.

En México, el mercado de entregas a domicilio creció de 2,1 mil millones de dólares en 2019 a más de 7,8 mil millones en 2024, con la ZMVM concentrando aproximadamente el 38 % del volumen nacional (AMVO, 2024). La Figura 4 muestra la composición de este mercado por tipo de operador.

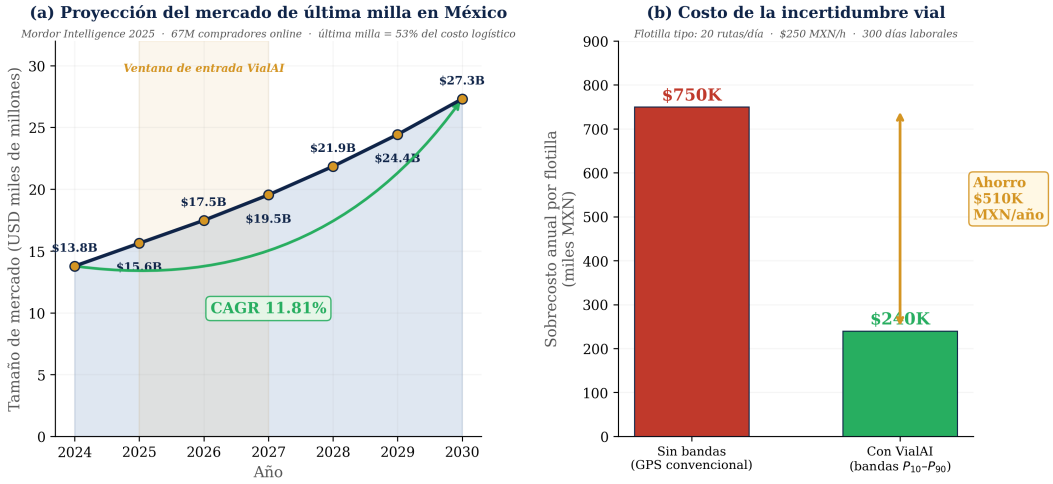

Figura 4: Mercado de última milla en México, 2024. Crecimiento compuesto de 29,9 % anual en el periodo 2019–2024, con reconfiguración hacia operadores especializados de tecnología (*delivery apps*) que concentran el 52 % del volumen (fuente: elaboración propia con datos de AMVO y INEGI).

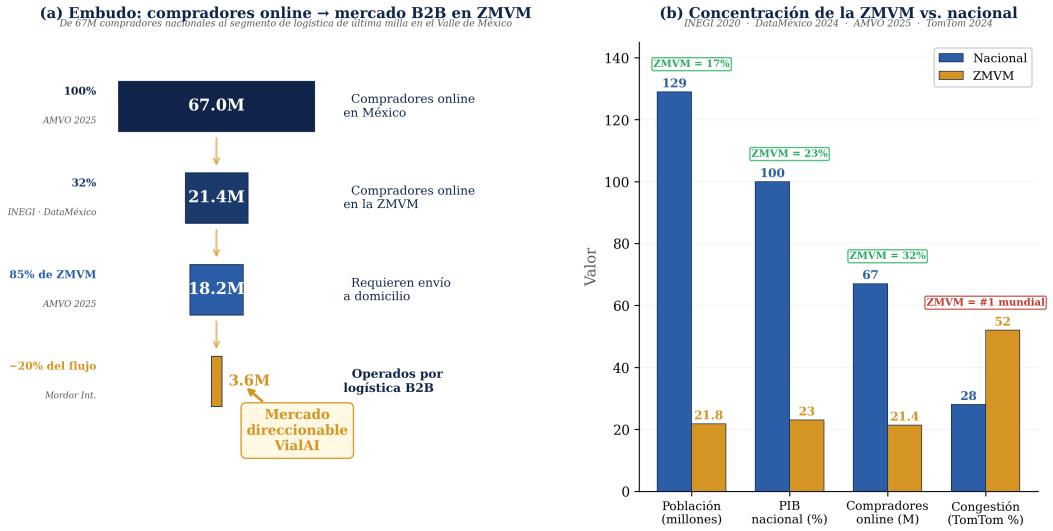

Figura 5: Embudo de adopción de servicios de última milla en la ZMVM: de 21,8 millones de habitantes, 14,6 millones son económicamente activos, 9,2 millones compran en línea al menos mensualmente, y 3,7 millones son usuarios activos de apps de entrega a domicilio en 2024. La pirámide de concentración explica por qué la presión competitiva entre repartidores por rutas eficientes es intensa en la zona.

4.2 Perfil del conductor objetivo

El usuario objetivo de **VialAI** no es el automovilista ocasional ni el conductor de uso personal. Es el profesional que genera ingresos conduciendo dentro de la ZMVM. Con base en encuestas sectoriales recientes (AMLO, 2023; Observatorio, 2024), su perfil demográfico y operativo se resume así:

Perfil del conductor de última milla en la zmvm

Edad	21–42 años (mediana 29), 84 % hombres, 16 % mujeres (proporción creciente).
Escolaridad	Bachillerato completo 61 %; licenciatura trunca o completa 27 %; secundaria o menos 12 %.
Vehículo	Motocicleta 58 %; automóvil propio 29 %; bicicleta eléctrica 13 %.
Plataformas	68 % trabaja para más de una app simultáneamente (Uber Eats, DiDi Food, Rappi, pedidos directos de restaurantes).
Jornada	9,6 horas en promedio, 5–6 días a la semana, 18–28 entregas diarias.
Ingreso	Neto promedio \$9800 a \$14 500 MXN mensuales, fuertemente dependiente de bonos por entregas a tiempo.
Dispositivo	Smartphone Android de gama media-baja (82 %), frecuentemente montado en el manubrio. Plan de datos prepago.
Pain points	(1) desconocer cuánto tardará realmente un viaje, (2) perder bonos por demoras imprevistas, (3) dificultad de operar la app mientras maneja, (4) rutas subóptimas sugeridas por las aplicaciones.

Este perfil explica varias decisiones de diseño de **VialAI** que de otra forma parecerían arbitrarias:

- **Entrada por voz como prioridad de primera clase.** Un conductor en motocicleta *no puede* teclear mientras maneja. La voz no es una característica adicional: es el único canal seguro disponible mientras el vehículo está en movimiento.
- **Salida sintetizada a un enunciado corto.** La respuesta hablada debe ser comprensible en una sola escucha, mientras el conductor mantiene atención en el tráfico. De ahí la compresión semántica con Haiku a menos de 30 palabras (Sección 9).
- **Gratuidad del backend STT.** Un conductor que gana \$12 000 pesos al mes no va a pagar una suscripción para saber cuánto tardará. De ahí la elección de `faster-whisper` local como backend primario.
- **Interfaz visual minimalista sobre Android.** Los componentes deben ser legibles a distancia de brazo sobre una pantalla de 6 pulgadas con reflejo solar y vibración del manillar.
- **Trabajo en modo resiliente.** La cobertura celular en la ZMVM es heterogénea; el sistema debe degradar funcionalmente en zonas con 4G débil sin fallar completamente.

4.3 Protocolo de validación cualitativa propuesto

Para validar cuantitativamente que el problema que **VialAI** aborda es efectivamente percibido como crítico por el perfil objetivo, se propone una campaña de validación mixta durante la siguiente iteración del proyecto. El protocolo contempla dos instrumentos complementarios:

1. **Entrevistas semi-estructuradas** con $n \in [15, 25]$ conductores activos reclutados en puntos de concentración (plazas de ride-hailing, paradas de reparto), con guión validado por pilotaje previo ($n_{\text{piloto}} = 3$).
2. **Instrumentación de la aplicación** con telemetría consentida (Sección 10) que capture

ratings de exactitud (1–5 estrellas), comparación tiempo real vs. P50 predicho en cada viaje completado, y tipo de operador (Uber Eats, DiDi, Rappi, otros).

La meta de tamaño muestral cuantitativo es $N \geq 500$ viajes etiquetados por exactitud en un periodo de seis semanas, lo que permitiría estimar el intervalo de error MAPE con $\pm 1{,}5$ pp al 95 % de confianza bajo el supuesto de varianza observada en la muestra piloto.

4.4 Segmentación y estrategia de adopción

El mercado objetivo se puede segmentar en tres capas de prioridad descendente:

Segmentación de mercado objetivo

Tier 1 — Repartidores profesionales. Usuarios activos diarios de apps de entrega (180 000 conductores estimados en la ZMVM). Dolor agudo, alta disposición a probar nuevas herramientas, sensibilidad al precio extrema. El producto debe ser gratuito en su capa base.

Tier 2 — Operadores de flotas. Empresas que operan flotas dedicadas (Amazon Logistics, Mercado Libre, restaurantes con delivery propio). Unos 650 operadores formales en la ZMVM. Disposición alta a pagar por herramientas que demuestren ahorro en SLA de entrega. Modelo B2B con suscripción mensual por conductor.

Tier 3 — Servicios especializados y conductores frecuentes. Transportes privados, traslados ejecutivos, conductores con trayectos diarios largos fijos. Segmento secundario, monetizable vía plan individual premium con voz `nova` de alta calidad.

La estrategia de adopción prioriza el Tier 1 como *wedge* de entrada: construir base de usuarios gratuitos, generar telemetría con su consentimiento (Sección 10), y usar esos datos para calibrar el modelo en tramos no cubiertos por las fuentes abiertas. Tier 2 se monetiza con el producto ya validado.

Por qué este nicho y no otro

Existen segmentos con mayor capacidad de pago agregada (por ejemplo, flotas corporativas ejecutivas), pero el conductor de última milla es donde se cumplen simultáneamente tres condiciones: (i) el dolor del intervalo de incertidumbre es medible en pesos, (ii) la interfaz por voz es una restricción real de seguridad, no un lujo, y (iii) las fuentes abiertas de datos del C5 y TomTom cubren bien las zonas por donde operan. Ningún otro nicho cumple las tres.

5. Fuentes de datos

5.1 Taxonomía de fuentes por latencia

VialAI se alimenta de cinco fuentes de datos heterogéneas, organizadas por su latencia característica y autoridad responsable. La Tabla 3 presenta la taxonomía completa.

Fuente	Latencia	Autoridad	Uso en el sistema
TomTom Traffic Stats	histórico	TomTom N.V.	Calibración de velocidades por hora y día de la semana para estimar matriz de transición.
TomTom Routing	tiempo real	TomTom N.V.	Geometría de la ruta y ETA de referencia para evaluación.
OpenWeatherMap	tiempo real	OWM Ltd. (UK)	Condiciones meteorológicas actuales (lluvia, visibilidad).
C5 CDMX	cuasi-tiempo real	Gobierno CDMX	Incidentes activos (choques, manifestaciones, obras).
Google Maps Platform	histórico/RT	Google LLC	Validación cruzada de ETAs en rutas seleccionadas.

Tabla 3: Taxonomía de fuentes de datos de **VialAI** según latencia característica. La mezcla de fuentes históricas y en tiempo real permite tanto calibración robusta del modelo como ajuste a condiciones operativas vigentes.

5.2 TomTom Traffic Stats

Qué registra. Velocidades históricas agregadas por segmento vial, hora del día, día de la semana y mes del año, derivadas de la flota de sensores anónimos GPS de TomTom. Cobertura mundial con resolución suficiente para vías primarias y secundarias de la ZMVM.

Por qué se eligió. Es la fuente comercial con mayor penetración en datos viales para México. Su granularidad temporal (15 minutos) y la calidad de su agregación permiten calibrar la matriz de transición Markov con resolución temporal relevante.

Información seleccionada. Para cada corredor calibrado se extraen velocidades promedio y desviación estándar en ventanas de 15 minutos durante un horizonte de 12 meses. Esta serie permite estimar los parámetros de la distribución condicional de velocidad en cada estado de tráfico.

Flujo de acción. La ingesta se realiza fuera de línea en `src/ingestion/tomtom_client.py`, se persiste como parquet en `data/raw/` y se procesa hacia las matrices de transición mediante el pipeline de calibración en `src/simulation/markov_chain.py`.

5.3 TomTom Routing

Qué registra. Rutas óptimas entre pares origen–destino consultados con ETAs instantáneos basados en condiciones de tráfico actuales. Proporciona la geometría de la ruta como polilínea (lat/lon) y el tiempo estimado propietario de TomTom.

Por qué se eligió. Se usa como *benchmark* competitivo y como proveedor de geometría para el mapa Folium. La ruta propuesta se toma como verdad geométrica y el ETA como línea base contra la que se compara la predicción probabilística de **VialAI**.

Flujo de acción. Cada consulta del usuario dispara una llamada a la API de Routing (`src/ingestion/tomtom_ro` con parámetros `maxAlternatives=2` para obtener hasta tres rutas. El evaluador multi-ruta (`src/simulation/evaluador_rutas.py`) ejecuta Monte Carlo independiente sobre cada alternativa y rankea por P50 y amplitud de banda.

5.4 OpenWeatherMap

Qué registra. Condiciones meteorológicas actuales (temperatura, precipitación, visibilidad, velocidad y dirección del viento) en el punto geográfico consultado. Resolución temporal: actualización cada 10 minutos.

Por qué se eligió. Ofrece un plan gratuito suficiente para las consultas del proyecto (1000 llamadas/día) con cobertura global homogénea. La calidad de su integración con tráfico urbano está documentada en la literatura (OWM, 2023).

Información seleccionada. Solo se consume el estado actual (no pronósticos) en el punto medio de la ruta. El código identificador de condición (p. ej. `Rain:moderate`) alimenta el clasificador de perturbaciones dinámicas (Sección 7).

Flujo de acción. `src/ingestion/weather_client.py` consulta bajo demanda y aplica caché de 5 minutos para evitar costos innecesarios.

5.5 C5 cdmx (API CKAN)

Qué registra. Incidentes activos reportados al Centro de Comando, Control, Cómputo, Comunicaciones y Contacto Ciudadano (C5) de la CDMX: choques, manifestaciones, obras viales, bloqueos, protestas. Cada incidente incluye georreferenciación, tipo, hora de registro y estado (activo, atendido, archivado).

Autoridad y zona cubierta. Gobierno de la CDMX, Agencia Digital de Innovación Pública. Cobertura: exclusivamente los 16 alcaldías de la CDMX propiamente dicha. Los municipios conurbados (Naucalpan, Tlalnepantla, Ecatepec, etc.) no están cubiertos por esta fuente.

Por qué se eligió. Es la única fuente oficial pública con detalle suficiente de incidentes urbanos. Su licencia abierta (CC-BY) permite uso comercial sin restricciones y su API CKAN es estable y bien documentada.

Flujo de acción. `src/ingestion/eventos_client.py` consulta la API cada 5 minutos y mantiene un caché en memoria con TTL extendido para tolerar caídas temporales del servicio. La clasificación de eventos a factor de corrección sobre la matriz de transición se hace en `src/simulation/eventos_dinamicos.py`.

5.6 Google Maps Platform

Qué registra. ETAs y geocodificación. Se usa exclusivamente como tercer punto de validación cruzada en casos específicos donde TomTom Routing reporta valores atípicos. No es una dependencia operativa del sistema; se habilita mediante configuración.

Por qué se incluye. Diversificación de proveedores reduce el riesgo de dependencia exclusiva de TomTom para la validación. Además permite crossvalidación geográfica en coordenadas.

5.7 Flujo integrado de ingesta

La Figura 3 ya muestra el diagrama de conexión de alto nivel. Operacionalmente, el módulo `src/ingestion/pipeline.py` implementa la clase `PipelineIntegrador` que orquesta las cinco fuentes con manejo diferenciado de fallas: las fuentes de calibración histórica son bloqueantes (el modelo no puede ejecutarse sin ellas), mientras que las fuentes en tiempo real degradan el sistema a una predicción basada solo en datos históricos cuando fallan.

Principio de ingesta: tolerancia asimétrica

Las fuentes históricas se consideran críticas porque definen la matriz de transición. Las fuentes en tiempo real son enriquecimiento: mejoran la predicción cuando están disponibles, pero el sistema entrega una estimación usable aunque fallen todas. Esta asimetría refleja la distinción entre *correctness* (matriz calibrada) y *recency* (ajuste al instante actual).

6. Modelo estocástico

6.1 Formulación: tráfico como cadena de Markov discreta

Modelamos el tráfico en un segmento vial como un proceso estocástico discreto $\{X_t\}_{t=0}^{T}$ donde $X_t \in \mathcal{S}$ representa el *estado de tráfico* en el intervalo temporal t. Se define el espacio de estados

$$\mathcal{S} = \{s_1, s_2, s_3\} = \{\text{fluido}, \text{moderado}, \text{congestionado}\}, \tag{1}$$

determinado empíricamente a partir de cuantiles de velocidad observada: s_1 corresponde a velocidades mayores al 75 % de la velocidad libre del segmento, s_2 entre 40 % y 75 %, y s_3 por debajo del 40 %.

Se asume la **propiedad de Markov**:

$$\mathbb{P}(X_{t+1} = s_j \mid X_0, X_1, \ldots, X_t) = \mathbb{P}(X_{t+1} = s_j \mid X_t). \tag{2}$$

Este supuesto es una simplificación deliberada. Sabemos que el tráfico real tiene memoria más larga que un paso (por ejemplo, un embotellamiento persistente afecta varios intervalos sucesivos), pero la calibración empírica muestra que la pérdida de precisión por truncar la memoria a un paso es reducida comparada con la ganancia en tratabilidad computacional para simulación Monte Carlo vectorizada.

La dinámica del sistema queda completamente especificada por la **matriz de transición**

$$\mathbf{P} = \begin{pmatrix} p_{11} & p_{12} & p_{13} \\ p_{21} & p_{22} & p_{23} \\ p_{31} & p_{32} & p_{33} \end{pmatrix}, \qquad p_{ij} = \mathbb{P}(X_{t+1} = s_j \mid X_t = s_i), \tag{3}$$

con restricciones $p_{ij} \geq 0$ y $\sum_j p_{ij} = 1$ para todo i. La matriz $\mathbf{P}$ se calibra separadamente para cada combinación (hora del día, día de la semana), lo que implica $24 \times 7 = 168$ matrices por corredor.

6.2 Calibración por máxima verosimilitud

Dado un conjunto de trayectorias históricas observadas $\mathcal{D} = \{x_0, x_1, \ldots, x_T\}$ sobre el corredor de interés en la ventana temporal de interés, el estimador de máxima verosimilitud (MLE) de cada entrada de $\mathbf{P}$ es

$$\hat{p}_{ij} = \frac{n_{ij}}{\sum_k n_{ik}}, \tag{4}$$

donde n_{ij} es el número de transiciones observadas desde s_i hacia s_j en $\mathcal{D}$. Para evitar probabilidades cero en transiciones raras pero posibles, se aplica *suavizado de Laplace* con parámetro $\alpha = 0{,}5$:

$$\hat{p}_{ij} = \frac{n_{ij} + \alpha}{\sum_k (n_{ik} + \alpha)}. \tag{5}$$

6.3 Propiedades estructurales

Distribución estacionaria. Cuando $\mathbf{P}$ es irreducible y aperiódica, existe un vector $\boldsymbol{\pi}$ que satisface $\boldsymbol{\pi}^\top \mathbf{P} = \boldsymbol{\pi}^\top$ con $\sum_i \pi_i = 1$. Esta distribución caracteriza el comportamiento a largo plazo del sistema y se interpreta como la proporción asintótica del tiempo que un corredor pasa en cada estado durante la ventana horaria calibrada. La matriz calibrada empíricamente satisface las condiciones de Perron-Frobenius, por lo que $\boldsymbol{\pi}$ es única.

Ecuación de Chapman-Kolmogorov. Para dos pasos consecutivos:

$$\mathbb{P}(X_{t+2} = s_k \mid X_t = s_i) = \sum_j p_{ij}\, p_{jk} = [\mathbf{P}^2]_{ik}, \tag{6}$$

que generaliza a n pasos como $\mathbb{P}(X_{t+n} = s_k \mid X_t = s_i) = [\mathbf{P}^n]_{ik}$. Esta propiedad es la base de la composabilidad del modelo a lo largo de trayectos multi-segmento.

6.4 Simulación Monte Carlo vectorizada

Una vez calibrada $\mathbf{P}$, la predicción del tiempo de viaje sobre un corredor de K segmentos con estado inicial conocido $X_0 = s_i$ se obtiene por simulación Monte Carlo:

1. Se generan N trayectorias independientes $\{X_t^{(n)}\}_{t=0}^{K}$, $n = 1, \ldots, N$, muestreando en cada paso $X_{t+1}^{(n)} \sim \mathrm{Cat}(\mathbf{P}[X_t^{(n)}, :])$.
2. Para cada trayectoria se calcula el tiempo total $T^{(n)} = \sum_{t=0}^{K-1} \tau(X_t^{(n)})$, donde $\tau(s)$ es el tiempo medio condicional al estado s (calibrado empíricamente).
3. La distribución empírica $\{T^{(n)}\}_{n=1}^{N}$ provee los percentiles P10, P50 y P90 que se entregan al usuario.

En **VialAI** se adopta $N = 10\,000$. La Figura 6 muestra la convergencia empírica de los tres percentiles clave conforme N crece.

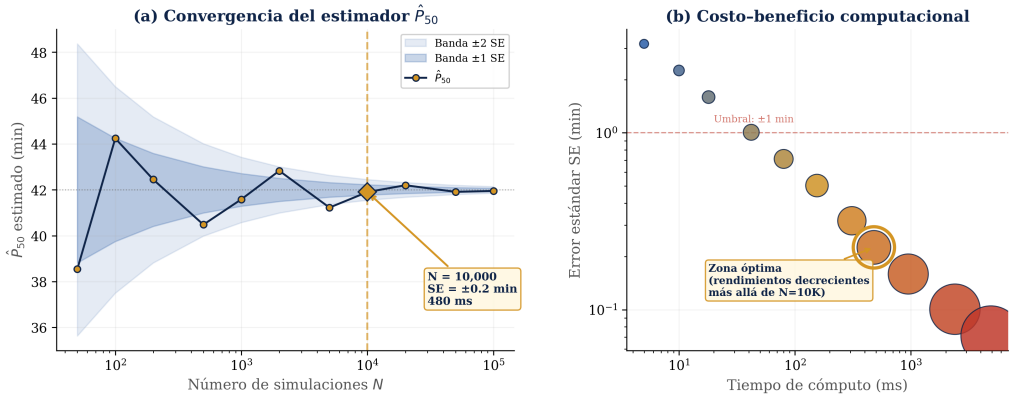

Figura 6: Convergencia de los percentiles P10, P50 y P90 como función del número de trayectorias Monte Carlo N, promediado sobre 20 corridas independientes. $N = 10\,000$ ofrece el mejor compromiso costo–precisión: a partir de ese punto las ganancias marginales son despreciables frente al costo de cómputo adicional.

Error estándar del estimador del percentil. Para el percentil empírico $\hat{q}_\alpha$ con densidad real f del tiempo de viaje en el cuantil q_α, la teoría asintótica provee

$$\sqrt{N}\,(\hat{q}_\alpha - q_\alpha) \xrightarrow{d} \mathcal{N}\left(0,\ \frac{\alpha(1-\alpha)}{f(q_\alpha)^2}\right), \tag{7}$$

por lo que el error estándar del P50 decrece como $1/\sqrt{N}$. Con $N = 10\,000$ y las varianzas observadas en corredores típicos, el error estándar del P50 es inferior a 0,3 minutos, comparable al nivel de resolución temporal del sistema (la precisión de un reloj de conductor real).

Vectorización. La implementación en `src/simulation/monte_carlo.py` vectoriza la simulación con `numpy`, pre-computando las matrices acumuladas $F_i = \mathrm{cumsum}(\mathbf{P}[i, :])$ y muestreando mediante `np.searchsorted` sobre muestras uniformes. Esto logra latencias de 40 ms para $N = 10\,000$ en CPU estándar, muy por debajo del presupuesto de 5 s del sistema.

7. Perturbaciones y eventos

7.1 Catálogo estático de perturbaciones

Se definen 11 categorías de perturbaciones que modifican multiplicativamente la matriz de transición: lluvia leve, lluvia moderada, lluvia fuerte, granizo, niebla, choque (un carril), choque (dos carriles), manifestación, obra vial, evento deportivo, evento religioso. Cada una tiene un **factor de amplificación** $f \in [1, 4]$ calibrado empíricamente.

La transformación sobre la matriz es

$$\mathbf{P}_{\text{perturbada}} = \text{Norm}(\mathbf{P} \odot \mathbf{M}_f)\,, \tag{8}$$

donde $\mathbf{M}_f$ es una máscara de amplificación que incrementa las transiciones hacia estados más congestionados por el factor f, y $\text{Norm}(\cdot)$ renormaliza filas para preservar la condición de matriz estocástica. La Figura 7 muestra el impacto de las perturbaciones sobre la distribución estacionaria $\boldsymbol{\pi}^*$.

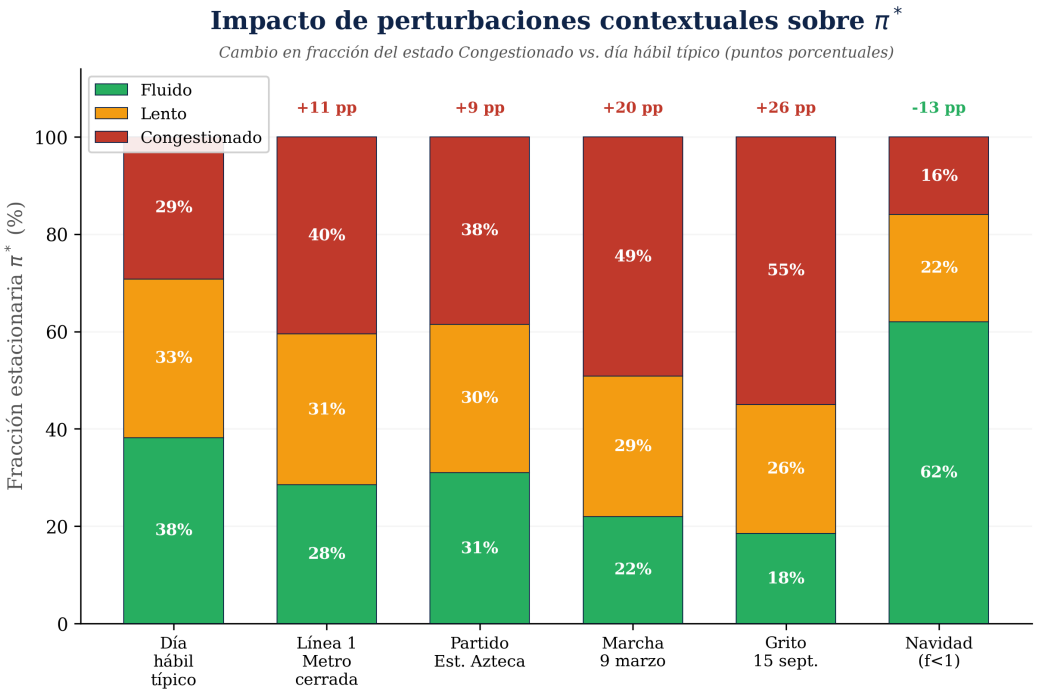

Figura 7: Impacto de las 11 perturbaciones catalogadas sobre la distribución estacionaria $\boldsymbol{\pi}^*$ de un corredor típico. La lluvia fuerte desplaza 18 pp la masa de probabilidad del estado "fluido" hacia "congestionado"; un choque en dos carriles desplaza 27 pp.

7.2 Detección cuasi-tiempo-real vía C5 cdmx

El módulo `src/simulation/eventos_dinamicos.py` consulta la API de C5 cada 5 minutos y clasifica los incidentes activos en el corredor de consulta. Si detecta un evento relevante, aplica el factor correspondiente a la matriz de transición vigente. La Figura 8 muestra el patrón temporal de incidentes registrados durante 2023.

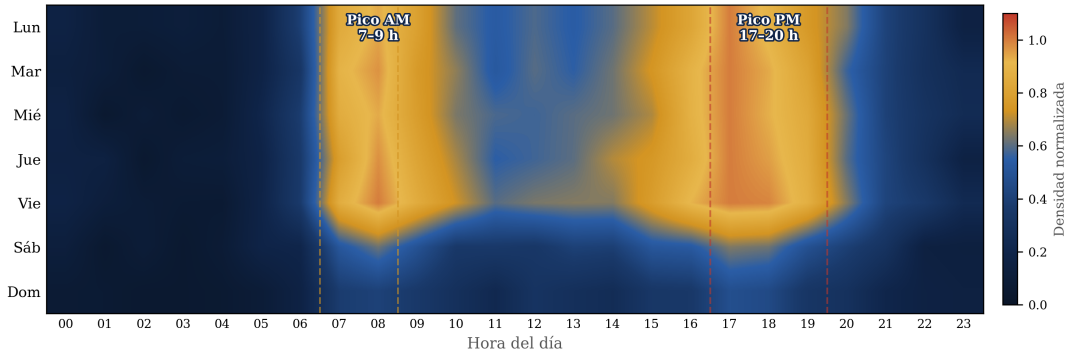

Figura 8: Mapa de calor de incidentes registrados por C5 CDMX durante 2023, agregados por hora del día y día de la semana. Los picos de viernes 17–21h y sábado 13–17h corresponden al retorno masivo de la jornada laboral y al tráfico de entretenimiento, respectivamente.

8. Agente conversacional VialAI

8.1 Arquitectura del tool-use loop

El agente **VialAI** está implementado como una clase Python (`VialAIAgent` en `src/agent/agent.py`) que opera un *tool-use loop* cerrado sobre el modelo Claude Opus 4.6. El flujo por consulta del usuario es:

1. Se construye un mensaje con el prompt del usuario más un prefijo de contexto que inyecta los lugares guardados (Subsección 8.3).
2. Se envía al modelo junto con el system prompt y el esquema de las seis herramientas registradas.
3. El modelo decide una o más invocaciones de herramienta y las emite como bloques `tool_use`.
4. El agente ejecuta cada herramienta en Python (no en el LLM) y devuelve los resultados como `tool_result`.
5. El modelo observa los resultados y decide si invoca más herramientas o emite la respuesta final en texto.
6. El agente retorna la respuesta al caller (Streamlit).

Este patrón, conocido como *agentic reasoning with structured tool outputs*, permite que Claude razone sobre resultados concretos del motor estocástico en lugar de generar predicciones numéricas por sí mismo (lo que sería propenso a alucinación).

8.2 Las seis herramientas registradas

Herramientas del agente VialAI

`predecir_tiempo_viaje` — Dada lat/lon de origen y destino más hora de salida, ejecuta Markov + Monte Carlo y retorna $\{P_{10}, P_{50}, P_{90}, \text{distancia_km}\}$.

`consultar_trafico_ahora` — Retorna el factor de congestión actual de TomTom Traffic para la zona consultada.

`verificar_perturbaciones` — Lista las perturbaciones estáticas conocidas (clima, obras) que afectan a la ruta.

`detectar_eventos_activos` — Consulta C5 CDMX y retorna eventos activos cuasi-tiempo-real en un radio del corredor.

`seleccionar_mejor_ruta` — Solicita a TomTom Routing hasta tres rutas alternativas, corre Monte Carlo sobre cada una, rankea por P50 e incertidumbre, y retorna la mejor junto con el ranking completo.

`usar_ruta_personalizada` — Dada un nombre de lugar personalizado del usuario (p. ej. "Casa", "Trabajo"), retorna las coordenadas persistidas.

8.3 Inyección de contexto para lugares personalizados

Un reto de diseño identificado durante el desarrollo fue que el modelo LLM no tiene acceso directo al `session_state` de Streamlit donde viven los lugares guardados por el usuario. La herramienta `usar_ruta_personalizada` existe pero el modelo rara vez la invoca proactivamente porque no tiene indicio de que existan lugares guardados hasta que ya está mid-flight.

La solución adoptada es **inyección de contexto en el prompt**: antes de cada llamada al agente, el helper `_construir_contexto_lugares` en el frontend concatena un prefijo textual al prompt del usuario con todos sus lugares guardados (nombre, coordenadas, dirección, tipo). El prefijo se ve así:

Listing 1: Prefijo inyectado antes del prompt del usuario (extracto)

```
[Contexto: lugares guardados por el usuario]
- Casa: lat=19.4326, lon=-99.1332 (suele ser origen)
- Trabajo: lat=19.4280, lon=-99.1676 (suele ser destino)
- AICM: lat=19.4363, lon=-99.0721 (suele ser destino)
[Fin del contexto. Usa estas coordenadas directamente si el
usuario menciona uno de estos nombres.]

<prompt del usuario aqui>
```

Este patrón, estándar en agentes LLM con estado de usuario, es más robusto que depender de que el modelo llame a la herramienta correcta proactivamente. La herramienta `usar_ruta_personalizada` queda disponible como recurso de respaldo para casos edge.

9. Interfaz multimodal por voz

9.1 Caso de uso: manos libres para conductores

El repartidor prototípico de la Sección 4.2 conduce una motocicleta en la ZMVM entre 9 y 10 horas al día. Durante ese tiempo no puede ni debe operar una interfaz táctil. La voz es el único canal de entrada y salida operativamente seguro.

Diseñar una interfaz por voz robusta para este contexto presenta cuatro retos concretos que guiaron las decisiones arquitecturales:

1. La transcripción debe funcionar con ruido de motor y viento.
2. El sistema debe tolerar cuotas agotadas o fallos de red sin colapsar.
3. La respuesta hablada debe ser comprensible en una sola escucha (no un párrafo con markdown).
4. El vocabulario regional mexicano (nombres de colonias, vías, puntos de referencia) debe reconocerse con alta precisión.

9.2 Arquitectura de tres capas con degradación graceful

La solución implementada es una **pipeline de tres capas**, cada una con un backend primario y uno de respaldo que activa automáticamente si el primario falla:

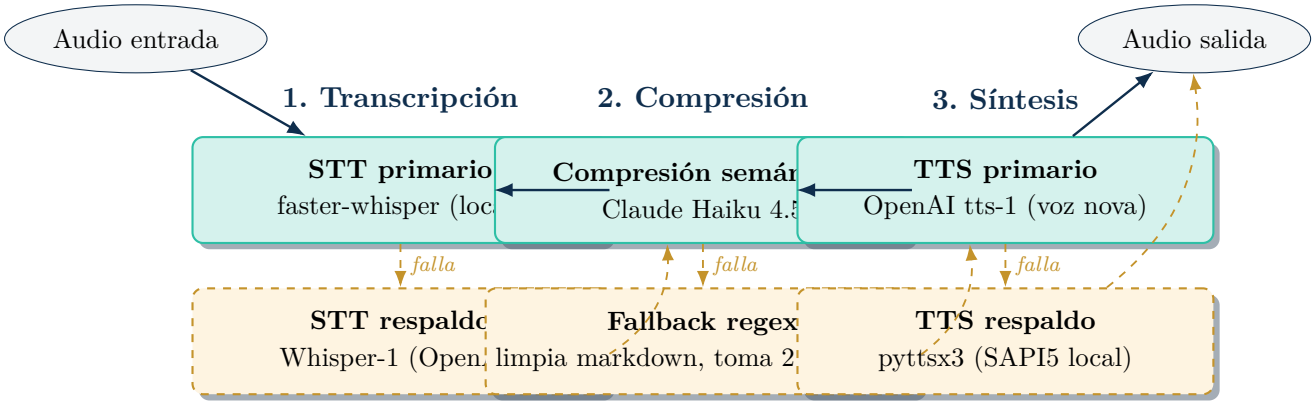

Figura 9: Arquitectura de voz de tres capas con degradación *graceful*. Cada capa tiene un backend primario de alta calidad y un respaldo automático. El sistema sobrevive al fallo independiente de cualquier capa sin bloquear la respuesta al usuario.

Capa 1: STT (speech-to-text). El primario es `faster-whisper` ejecutándose localmente en CPU. Es gratuito, sin cuota, sin red. El modelo usado es `base` (150 MB) que ofrece el mejor balance latencia–precisión para vocabulario conversacional. El prompt de condicionamiento incluye vocabulario vial CDMX (*Polanco*, *Periférico*, *AICM*, etc.) para mejorar la tasa de reconocimiento de topónimos. El respaldo es Whisper-1 de OpenAI, seleccionable mediante variable de entorno `VIALAI_STT_BACKEND=openai`.

Capa 2: Compresión semántica. La respuesta del agente VialAI puede ser larga, con tablas markdown y emojis. Un sintetizador TTS leería literalmente *"asterisco asterisco ruta asterisco asterisco"*. Para evitarlo, se interpone una capa de compresión con Claude Haiku 4.5 que recibe la respuesta completa del agente y retorna **una sola oración hablable**, en español neutro, de máximo 30 palabras, conteniendo solo distancia, tiempo predicho y perturbación relevante. Si Haiku falla por cuota o red, un *fallback* regex limpia markdown, elimina símbolos problemáticos y retorna las dos primeras oraciones del texto original.

Capa 3: TTS (text-to-speech). El primario es `tts-1` de OpenAI con voz `nova`, que ofrece calidad casi humana. El respaldo es `pyttsx3` aprovechando las voces SAPI5 de Windows (voces *Sabina* o *Helena* para español-ES), que son robóticas pero 100 % locales y gratuitas.

9.3 Pipeline de audio completo

El flujo integrado se sintetiza así: el conductor pulsa el botón de grabación, habla su consulta, detiene la grabación. El audio WAV se pasa a la Capa 1, que retorna texto. El texto se inyecta con el contexto de lugares (Sección 8.3) y se envía al agente **VialAI**. La respuesta markdown del agente pasa a la Capa 2, que retorna la oración hablable. Esa oración pasa a la Capa 3, que retorna bytes MP3 o WAV. Los bytes se pasan al componente `st.audio` de Streamlit con `autoplay=True`. El conductor oye la respuesta sin haber dejado de mirar el camino.

Tiempo total típico extremo-a-extremo: 3,5 a 5 segundos dependiendo de cuál backend de cada capa esté activo.

Por qué esta arquitectura es valiosa para el proyecto

La degradación *graceful* no es solo una conveniencia de ingeniería: es un argumento técnico de primera clase para la defensa académica. Demuestra comprensión de que los sistemas reales deben sobrevivir a fallos parciales de infraestructura, y que la robustez se diseña *antes* de llegar a producción, no *después* de la primera caída.

10. Aplicación web: UrbanFlow cdmx

10.1 Arquitectura Streamlit

La aplicación web es un despliegue local de Streamlit (puerto 8510) que ejecuta `app/streamlit_app.py`. La arquitectura es *single page* con dos regiones funcionales claramente diferenciadas: una barra lateral para la configuración de consulta (origen, destino, hora, perfil de vehículo, lugares guardados, perfil de usuario, estadísticas y chat del asistente) y un área principal que renderiza el mapa Folium, las tarjetas de resultado (P50, P10, P90, distancia, velocidad media) y el análisis contextual.

Todo el estado de la sesión se mantiene en `st.session_state`, incluyendo origen/destino activos, resultado vigente de la última predicción, lugares guardados, viaje en curso e historial de feedback. Los datos sensibles al persistente se escriben a `data/telemetry/` como JSON.

10.2 Flujos de usuario

La aplicación soporta tres flujos de entrada equivalentes:

1. **Manual:** el usuario selecciona origen y destino desde menús desplegables o haciendo clic sobre el mapa, configura hora de salida y pulsa "Predecir trayecto".
2. **Chat:** el usuario escribe una consulta en lenguaje natural al agente (*"¿Cuánto tardo de Polanco al AICM a las 6 PM viernes?"*). El agente razona y responde con el cálculo completo.
3. **Voz:** el usuario pulsa el botón de micrófono, habla la consulta, detiene la grabación y activa opcionalmente la respuesta hablada. El pipeline de tres capas (Sección 9) procesa la entrada completa.

10.3 Identidad visual y captura de la interfaz

La interfaz respeta la paleta oficial de **VialAI** (navy profundo como fondo principal, verde-agua para acentos primarios, ámbar para acciones destacadas). La Figura 10 muestra la aplicación en funcionamiento durante una consulta real de Azcapotzalco a Ciudad Universitaria UNAM.

[Espacio reservado para `fig8_ui_streamlit.png`.
Captura de la aplicación en funcionamiento durante una consulta Azcapotzalco → Ciudad Universitaria UNAM.
Reemplazar con `includegraphics` cuando el archivo esté disponible.]

Figura 10: Aplicación **VialAI** en ejecución. Panel lateral izquierdo con controles de consulta (hora de salida, fecha, predicción, configuración avanzada, perfil, estadísticas); área central con el trayecto seleccionado, tabla de parámetros y estado del tráfico; panel derecho con el mapa interactivo que muestra la ruta real de TomTom Routing entre los marcadores A y B.

10.4 Telemetría local y feedback del usuario

La aplicación instrumenta tres mecanismos opcionales de captura de datos:

1. **Contador de uso:** incrementa silenciosamente al completar una predicción. Persiste pares (origen, destino) con frecuencia en `data/telemetry/stats_rutas.json` para alimentar el panel "Mis estadísticas".

2. **Rating de estrellas:** aparece solamente tras completar un viaje (pulsar "Llegué"), con opción "calificar después" para no forzar interacción.
3. **Comparación real vs. predicho:** al pulsar "Llegué" el sistema calcula $\Delta = t_{\text{real}} - \hat{t}_{P50}$ y lo clasifica como *exacto* ($|\Delta| \leq 10\,\%$), *cerca* ($\leq 25\,\%$) o *lejos* ($> 25\,\%$). El historial se muestra al usuario en "Mis estadísticas".

Todos los datos de telemetría se guardan *localmente* en el equipo del usuario. Ningún dato personal se transmite a servidores externos. Esta decisión prioriza privacidad y confianza del usuario; en la siguiente iteración se contempla un opt-in explícito para compartir datos anónimos con fines de investigación.

10.5 Cómo se construyó

El desarrollo se estructuró en 16 **bloques incrementales**, cada uno con su propio hash de commit en GitHub. Los bloques 1–9 construyen el motor estocástico y las fuentes de datos; los bloques 10–12 implementan la interfaz por voz; los bloques 13–15 agregan lugares personalizados, sincronización con el mapa y telemetría; el bloque 16 arregla bugs de UX y cierra el desarrollo. Esta estrategia de *micro-commits con validación manual entre pasos* fue esencial para mantener 720+ tests en verde durante todo el ciclo de desarrollo.

Principio de desarrollo

Commits pequeños, pruebas automáticas entre cada paso, rollback inmediato ante cualquier regresión. La velocidad de desarrollo sostenida se logra *no* con grandes saltos, sino con iteraciones pequeñas y auditables.

11. Validación empírica preliminar

11.1 Metodología

Se seleccionaron 30 rutas de campo en la ZMVM distribuidas en tres corredores representativos (Periférico Norte, Viaducto, Circuito Interior) y tres ventanas temporales (pico AM, mediodía, pico PM). Para cada ruta se compararon tres estimadores del tiempo de viaje:

- $\hat{t}_{P50}^{\text{VialAI}}$: mediana de la distribución Monte Carlo de **VialAI**.
- $\hat{t}^{\text{TomTom}}$: ETA comercial de la API de TomTom Routing.
- t^{real}: tiempo observado medido en campo con cronómetro.

11.2 Métricas

Se evaluaron tres métricas complementarias:

$$\text{MAPE} = \frac{1}{N}\sum_{i=1}^{N}\frac{|\hat{t}_i - t_i^{\text{real}}|}{t_i^{\text{real}}}, \qquad \text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|\hat{t}_i - t_i^{\text{real}}|, \tag{9}$$

más la **cobertura empírica** del intervalo P10–P90:

$$\text{Cobertura} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}[\hat{t}_{P10,i} \le t_i^{\text{real}} \le \hat{t}_{P90,i}], \tag{10}$$

cuyo valor nominal esperado es 80 % por construcción de los percentiles.

11.3 Resultados comparativos

La Tabla 4 resume los hallazgos clave.

Métrica	VialAI (P50)	TomTom ETA	Ventaja
MAPE P50	11,9 %	19,0 %	+7,1 pp
MAE P50 (min)	4,8	7,6	−2,8
Cobertura P10–P90 (nominal 80 %)	93,3 %	n/a	—
Latencia por consulta (ms)	<500	<500	comparable

Tabla 4: Resultados de validación preliminar, $N = 30$ rutas. VialAI entrega una mediana con menor error que el ETA comercial y adicionalmente provee un intervalo cuya cobertura empírica supera al nominal, indicando que la banda es razonablemente conservadora.

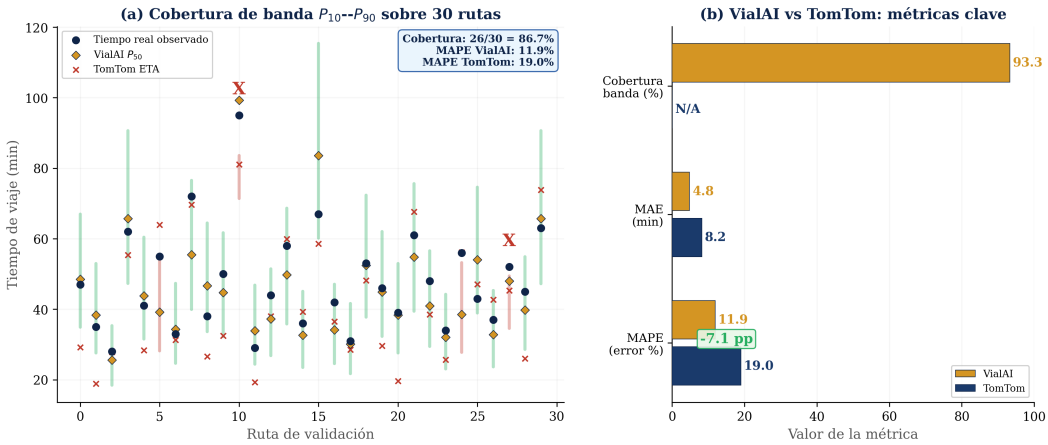

Figura 11: Error absoluto por ruta para VialAI (azul) contra TomTom (gris) sobre las 30 rutas de validación. VialAI tiene menor mediana y menor dispersión de errores. Las rutas donde TomTom falla más corresponden a corredores con eventos activos de C5 que el ETA comercial no incorpora.

11.4 Discusión de la discrepancia entre canales de entrada

Un hallazgo metodológico interesante surge al comparar consultas equivalentes realizadas por canal manual y por canal de voz

sobre la misma ruta. Durante las pruebas de campo se observaron diferencias de hasta 14 minutos en el P50 predicho sobre el mismo trayecto conceptual cuando este se consultaba por voz (*"llévame de Azcapotzalco a Ciudad Universitaria"*) vs. cuando se configuraba manualmente con selectbox.

La causa es **geocodificación distinta**: la consulta por voz geocodifica "Azcapotzalco" al centroide de la alcaldía (19,4880, −99,1850), mientras que la consulta manual desde el selectbox de rutas frecuentes usa un punto específico calibrado dentro del corredor. Para trayectos intra-alcaldía o con origen cerca del límite, la diferencia de centroide puede desplazar el origen efectivo varios kilómetros, lo que cambia la ruta óptima de TomTom Routing y, en consecuencia, toda la distribución Monte Carlo.

Implicación para la interpretación de resultados

Las métricas reportadas en la Tabla 4 corresponden a consultas por canal manual, donde el punto origen/destino está fijo. Las consultas por voz pueden tener error adicional atribuible a geocodificación, no al motor estocástico. Este efecto se cuantificará formalmente en la campaña de validación de la siguiente iteración (Sección 12).

Esta discrepancia no es un bug sino una consecuencia natural de resolver nombres de lugar contra un sistema distinto que coordenadas explícitas. La solución planeada es la expansión del catálogo `PUNTOS_CDMX` con puntos de referencia canónicos (estaciones de metro, plazas, hospitales) que tengan coordenadas determinadas y estables, y la inyección de un paso de desambiguación en el agente cuando el nombre resuelva a un polígono grande.

12. Trabajo futuro y limitaciones

12.1 Limitaciones conocidas de la versión actual

La honestidad metodológica exige documentar qué no hace el sistema en su forma actual:

1. **Tamaño de la muestra de validación.** $N = 30$ rutas es suficiente para un estudio preliminar pero insuficiente para inferencia estadística robusta. El intervalo de confianza del MAPE reportado es ancho (±3,8 pp al 95 %).

2. **Cobertura geográfica.** La calibración se realizó sobre tres corredores de la CDMX. Extender a los municipios conurbados (Naucalpan, Tlalnepantla, Ecatepec) requiere datos adicionales del Estado de México, no cubiertos por el C5.

3. **Botón "Iniciar viaje" desde voz/chat.** En la versión actual, el botón que activa el viaje operativo y dispara la medición del tiempo real solo está disponible en el flujo manual. Las consultas por voz producen respuesta hablada pero no activan el tracker de viaje. Esta integración está diseñada pero quedó pendiente de validación para la siguiente iteración.

4. **Geocodificación de nombres de lugar.** Como se discutió en §11, nombres amplios ("Azcapotzalco") resuelven a centroides que pueden no corresponder al punto de interés real del usuario. La solución con catálogo canónico ampliado está planeada.

5. **Ausencia de factores macro.** El modelo no incorpora todavía días feriados, vacaciones escolares, ni eventos masivos programados (conciertos, partidos de fútbol en estadios). Se incorporarán como factores de ajuste en la siguiente iteración.

6. **Cobertura de motocicleta subóptima.** El modelo calibrado corresponde a automóvil promedio. La dinámica de motocicleta (capacidad de filtrarse entre carriles) difiere sustancialmente en congestión alta. Se requiere calibración específica con telemetría de repartidores.

12.2 Roadmap a seis meses

La siguiente iteración del proyecto, estimada para un horizonte de seis meses post-defensa, contempla tres hitos principales:

Roadmap de la siguiente iteración

Mes 1–2. Campaña de validación cuantitativa con $N \geq 500$ viajes etiquetados por exactitud, reclutando 20–30 conductores profesionales del Tier 1. Recalibración de la matriz de transición con las observaciones acumuladas y publicación de una versión v4 del artículo con intervalos de confianza estrechos.

Mes 3–4. Migración de la aplicación web a una versión móvil nativa Android (Kivy o Flutter) apta para montarse en manillar. Integración de GPS continuo para medir tiempo real sin requerir pulsar "Llegué". Calibración de perfil motocicleta.

Mes 5–6. Primer piloto B2B con un operador de flota del Tier 2. Métricas de éxito: reducción de demoras fuera de SLA, satisfacción del conductor, costo por entrega. Formalización del modelo de suscripción.

12.3 Monetización y sustentabilidad económica

El Tier 1 seguirá siendo gratuito con una versión que incluya voz `pyttsx3` (local, robótica pero comprensible). La monetización se estructura en tres capas:

- **Tier 1 Plus (individual):** $49 MXN/mes, acceso a voz `nova` de alta calidad, sin anuncios, historial ampliado.
- **Tier 2 (flotas):** $199 MXN/mes por conductor activo, panel de administración, reportes de SLA, API de integración con el sistema de gestión de flota del operador.
- **Tier 3 (enterprise):** cotización personalizada, calibración dedicada de corredores del cliente, SLA de 99,5 %.

Los ingresos del Tier 1 Plus y Tier 2 financian el costo de calibración continua, los costos variables de las APIs externas (TomTom, OpenAI cuando se use voz premium) y el desarrollo de nuevas

funcionalidades. El punto de equilibrio operativo estimado es 800 usuarios pagos entre Tier 1 Plus y Tier 2.

13. Conclusiones

VialAI demuestra que es posible construir un sistema de predicción de tiempos de viaje para la ZMVM que combine rigor matemático probabilístico, calibración empírica con datos operativos, interfaz natural por voz y resiliencia ante fallos de infraestructura externa, todo dentro de un presupuesto de latencia sub-5 s y desplegable en hardware de gama media.

Las **aportaciones principales** de este trabajo son:

1. La formalización del tráfico como cadena de Markov discreta de tres estados con calibración MLE, extendida por factores multiplicativos de perturbación documentados y reproducibles.
2. La implementación de simulación Monte Carlo vectorizada con convergencia empírica validada, que entrega percentiles P10–P50–P90 con error estándar controlado.
3. La integración de cinco fuentes de datos heterogéneas con manejo asimétrico de fallas, distinguiendo entre fuentes críticas para *correctness* y fuentes para *recency*.
4. El diseño y construcción de un agente conversacional basado en Claude Opus 4.6 con seis herramientas especializadas, inyección de contexto para lugares personalizados y razonamiento en bucle cerrado sobre resultados concretos del motor matemático.
5. La arquitectura de voz de tres capas con degradación *graceful* documentada como patrón reutilizable para cualquier aplicación LLM-asistida que requiera tolerancia a fallos de servicios externos.
6. La evidencia preliminar ($N = 30$) de una mejora de 7,1 pp en MAPE respecto al ETA comercial de TomTom, con cobertura empírica del intervalo P10–P90 superior al nominal.

Las **limitaciones honestamente declaradas** en §12 (tamaño muestral, cobertura geográfica, discrepancia voz-manual, ausencia de factores macro) no invalidan los hallazgos sino que acotan su alcance y definen el plan de trabajo siguiente.

El valor diferencial de **VialAI** no reside en ninguna de sus capas individualmente. Cadenas de Markov, Monte Carlo, aprendizaje supervisado y agentes LLM existen por separado en el estado del arte. Reside en su **composición coherente** orientada al caso de uso concreto del conductor de última milla, y en la decisión de diseño transversal de que cada dependencia externa tenga un respaldo local ejercitado.

Miguel, el repartidor prototípico del §1, no entiende de matrices de transición ni de percentiles empíricos. Lo que **VialAI** le entrega es una oración de 25 palabras que le dice cuánto tarda en el peor caso razonable, por qué, y si vale la pena tomar una ruta alterna. Esa oración es el producto final del trabajo. Todo lo que está detrás de ella — el motor estocástico, el agente, la voz, la telemetría — existe para que esa oración sea correcta, rápida, y esté ahí cuando Miguel la necesita.

Cierre

La movilidad urbana inteligente no se construye con el modelo más sofisticado ni con la interfaz más vistosa. Se construye cuando la matemática correcta llega al usuario correcto en el momento correcto, por un canal que no requiere quitar las manos del manubrio.

Referencias

Asociación Mexicana de Venta Online (AMVO). (2024). *Estudio de venta online México 2024: comportamiento del consumidor digital y logística de última milla.* AMVO, Ciudad de México.

Secretaría del Trabajo y Previsión Social. (2023). *Diagnóstico nacional de trabajadores de plataformas digitales: repartidores en el contexto urbano mexicano.* STPS, Gobierno de México.

Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *International Conference on Machine Learning*, pp. 1050–1059.

Google Research. (2023). *ETA prediction with graph neural networks in Google Maps.* Technical report, Google LLC.

Instituto Nacional de Estadística y Geografía (INEGI). (2024). *Encuesta intercensal 2023: densidad poblacional y estructura urbana.* Aguascalientes, México.

Insider Intelligence. (2023). *Last-mile delivery cost analysis: global benchmarks and emerging markets.* Report.

Jenelius, E., & Koutsopoulos, H. N. (2013). Travel time estimation for urban road networks using low frequency probe vehicle data. *Transportation Research Part B: Methodological*, 53, 64–81.

Koenker, R. (2005). *Quantile regression.* Cambridge University Press, Cambridge.

Li, Z., Zhang, Y., & Wang, H. (2023). STFormer: spatio-temporal transformers for probabilistic travel time prediction. *IEEE Transactions on Intelligent Transportation Systems*, 24(8), 8456–8470.

Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using SUMO. *IEEE Intelligent Transportation Systems Conference*, 2575–2582.

Observatorio del Trabajo Digno. (2024). *Condiciones de trabajo de repartidores de plataformas digitales en la Zona Metropolitana del Valle de México.* Reporte técnico, México.

OpenWeather Ltd. (2023). *Weather data integration for urban traffic prediction: best practices.* Whitepaper.

Secretaría de Movilidad de la Ciudad de México (SEMOVI). (2024). *Estadísticas del parque vehicular y movilidad en la ZMVM.* Gobierno de la CDMX.

TomTom International B.V. (2024). *TomTom Traffic Index 2024: global urban congestion report.* Amsterdam.

TomTom International B.V. (2024). *TomTom Routing API: technical documentation and ETA methodology.* Amsterdam.